

Acceleration, Momentum, and Stochastic Gradient

What fixes slow gradient descent — and what doesn't

Dr. Robert Hildebrand

Spring 2026

Grado Department of Industrial & Systems Engineering
Virginia Tech

Today — three ideas, three fixes

The starting point

Plain gradient descent is *slow* on most problems you care about. Not slow because computers are slow — slow because the method leaves information on the table.

Three fixes, three different diagnoses of “slow”

1. **Heavy-ball momentum** — fixes oscillation on ill-conditioned problems
2. **Nesterov acceleration** — achieves the *provably optimal* first-order rate
3. **Stochastic gradients** — fixes the “full gradient is too expensive” regime

All three are still active research today. But the 1983 version of Nesterov’s accelerated method is exactly what FISTA uses, Adam is momentum-plus-Jacobi, and heavy-ball is what you implement when you type `momentum=0.9` in PyTorch. The *ideas* are stable.

Roadmap

Part 1 — Why Gradient Descent Is Slow

Part 2 — Heavy-Ball Momentum (Polyak, 1964)

Part 3 — Nesterov Acceleration (1983)

Part 4 — Stochastic Gradient Descent

Part 5 — Combining the Ideas

Gradient descent and the condition number

The iteration (unit step, exact line search aside)

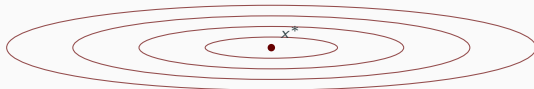
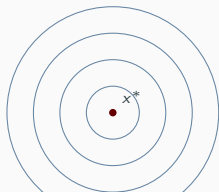
$$x_{k+1} = x_k - \alpha \nabla f(x_k).$$

For L -smooth f : pick $\alpha = 1/L$. For μ -strongly-convex f too:

$$f(x_k) - f^* \leq \left(1 - \frac{1}{\kappa}\right)^k (f(x_0) - f^*), \quad \kappa = L/\mu.$$

What κ means geometrically

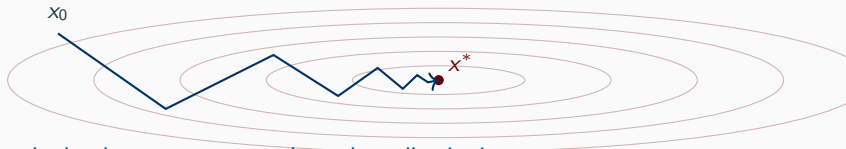
The Hessian's largest-to-smallest eigenvalue ratio. Level sets are *ellipsoids with axes in proportion $\sqrt{\lambda_i}$* . Large $\kappa \iff$ elongated bowl.



What GD actually does on a narrow valley

The zigzag, visualized

The gradient at any point is *perpendicular* to the level set there. On an elongated ellipse, that direction is almost orthogonal to the direction of x^* .



each step is perpendicular to the level set — progress along the valley is tiny

The quantitative cost

At $\kappa = 10^4$, the contraction factor is $1 - 10^{-4}$. To halve the error you need $\approx 7,000$ iterations. And $\kappa = 10^4$ is *routine* in ML and control.

A concrete 2D example — the one we'll return to

The running test problem

$$f(x) = \frac{1}{2} x^\top A x, \quad A = \text{diag}(1, 100).$$

Smooth ($L = 100$), strongly convex ($\mu = 1$), $\kappa = 100$. Optimum is $x^* = 0$. Level sets: ellipses with axes 10 : 1.

```
# GD on the 2D ill-conditioned quadratic
A = np.diag([1.0, 100.0])
alpha = 1.0 / 100.0          # = 1/L
x = np.array([10.0, 1.0])
hist = [x.copy()]
for _ in range(200):
    g = A @ x
    x = x - alpha * g
    hist.append(x.copy())
errs = [0.5 * xk @ A @ xk for xk in hist]
```

What you'll see in the terminal

After 200 iterations GD's suboptimality is $\approx 10^{-2}$. The x_1 -component (slow eigenvalue = 1)

Roadmap

Part 1 — Why Gradient Descent Is Slow

Part 2 — Heavy-Ball Momentum (Polyak, 1964)

Part 3 — Nesterov Acceleration (1983)

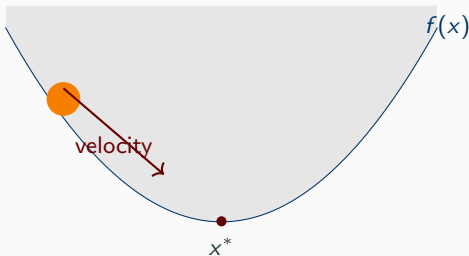
Part 4 — Stochastic Gradient Descent

Part 5 — Combining the Ideas

The physical picture

Why a marble beats gradient descent in a narrow bowl

A marble released in an elongated bowl accumulates velocity along the valley floor. It doesn't reverse direction every time the slope flips sign, because it has *inertia*. Gradient descent has none.



What it does mathematically

Add a “momentum” term: the update includes a fraction of the previous displacement. The algorithm keeps moving in directions that have been consistently productive, and smooths out the zigzag.

The heavy-ball update

Polyak's recursion

$$x_{k+1} = x_k - \alpha \nabla f(x_k) + \beta (x_k - x_{k-1})$$

Equivalently, with velocity $v_k = x_k - x_{k-1}$:

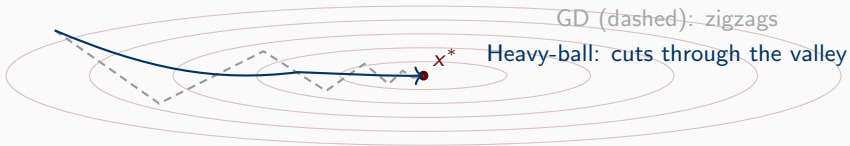
$$v_{k+1} = \beta v_k - \alpha \nabla f(x_k), \quad x_{k+1} = x_k + v_{k+1}.$$

```
# Heavy-ball in four lines  
v = np.zeros_like(x)  
for _ in range(max_iter):  
    v = beta * v - alpha * grad(x)  
    x = x + v
```

What the three pieces do

- $-\alpha \nabla f(x_k)$: the ordinary gradient step.
- βv_k : decaying memory of past steps.

Heavy-ball vs. GD on the narrow valley



What just happened in the picture

Momentum smooths out the cross-valley oscillation and accumulates consistent progress along the valley floor. On a diagonal quadratic, the accumulated step size along the slow direction grows by a factor of roughly $1/(1 - \beta)$ compared to GD.

Optimal tuning on quadratics — and the $\sqrt{\kappa}$ rate

Polyak's optimal constants (for quadratic f)

$$\alpha^* = \frac{4}{(\sqrt{L} + \sqrt{\mu})^2}, \quad \beta^* = \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^2.$$

Why it's remarkable

With these constants, the contraction rate per iteration is

$$\rho_{\text{HB}} = \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \approx 1 - \frac{2}{\sqrt{\kappa}}.$$

Compare $\rho_{\text{GD}} \approx 1 - 1/\kappa$. At $\kappa = 10^4$: GD needs $\approx 7,000$ iterations to halve the error; HB needs ≈ 70 . **Square root of the condition number.**

The proof is for quadratics. On *non-quadratic* convex f , heavy-ball does *not* have a clean global convergence guarantee with these constants — there are smooth convex counterexamples where it diverges. In practice it still works on many problems; theoretically, you want the next idea.

Roadmap

Part 1 — Why Gradient Descent Is Slow

Part 2 — Heavy-Ball Momentum (Polyak, 1964)

Part 3 — Nesterov Acceleration (1983)

Part 4 — Stochastic Gradient Descent

Part 5 — Combining the Ideas

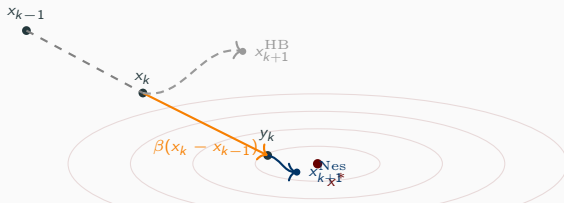
One small change — where you evaluate the gradient

Heavy-ball, restated

1. Predict: extrapolate along momentum to $y_k = x_k + \beta(x_k - x_{k-1})$.
2. Correct: take a gradient step *from* x_k .

Nesterov's fix

1. Predict: $y_k = x_k + \beta(x_k - x_{k-1})$.
2. Correct: take a gradient step *from* y_k .



Nesterov / FISTA — the full update

The t_k schedule (for the $O(1/k^2)$ rate)

$$t_0 = 1, \quad t_{k+1} = \frac{1}{2} \left(1 + \sqrt{1 + 4t_k^2} \right), \quad \beta_k = \frac{t_k - 1}{t_{k+1}}.$$

Note $\beta_k \rightarrow 1$ as $k \rightarrow \infty$: more momentum later, when you're oscillating near the optimum.

```
x_prev, y, t = x0.copy(), x0.copy(), 1.0
for k in range(max_iter):
    x_new = y - alpha * grad(y)           # gradient from lookahead
    t_new = 0.5 * (1.0 + np.sqrt(1.0 + 4.0*t*t)) # Nesterov schedule
    beta = (t - 1.0) / t_new              # adaptive momentum
    y     = x_new + beta * (x_new - x_prev)  # extrapolate
    x_prev, t = x_new, t_new
```

What to appreciate

One gradient evaluation per iteration, same as GD. Two state variables: x (iterate) and y (lookahead). *Adaptive* momentum, no manual tuning of β needed — the schedule picks it for you.

Nesterov's theorem — the rate

Theorem (Nesterov 1983; Beck–Teboulle 2009 for FISTA)

For L -smooth convex f with minimizer x^* , Nesterov's method with step size $\alpha = 1/L$ satisfies

$$f(x_k) - f^* \leq \frac{2L \|x_0 - x^*\|_2^2}{(k+1)^2}.$$

This is the **optimal rate** among first-order methods — Nemirovski–Yudin (1983) proved $\Omega(1/k^2)$ is a matching lower bound for any method that only queries f and ∇f .

Strongly convex version

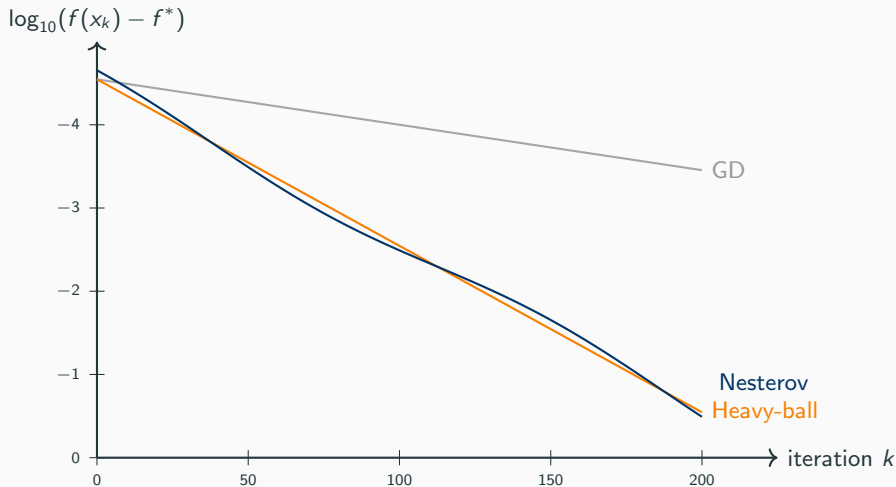
For μ -strongly-convex on top of L -smooth, the rate becomes

$$f(x_k) - f^* \leq \left(1 - \sqrt{\frac{1}{\kappa}}\right)^k \cdot C.$$

Same $\sqrt{\kappa}$ improvement as heavy-ball — but now with a *proof* that doesn't require the problem to be quadratic.

Optimal means: no first-order method can do better *in the worst case*. Better rates exist

What acceleration looks like — the convergence plot



What the picture shows on a moderately ill-conditioned problem

FISTA — Nesterov when your f isn't smooth

Composite problem structure

$$\min_x f(x) + g(x), \quad f \text{ smooth convex}, \quad g \text{ closed convex (nonsmooth)}.$$

Think LASSO: $f = \frac{1}{2}\|Ax - b\|^2$ (smooth), $g = \lambda\|x\|_1$ (nonsmooth).

The swap: gradient step $\rightarrow$ prox-gradient step

$$x_{k+1} = \underbrace{\text{prox}_{\alpha g}}_{\text{cheap if } g \text{ is nice}} (y_k - \alpha \nabla f(y_k)).$$

For $g = \lambda\|x\|_1$ this is componentwise soft-thresholding. For $g = \iota_C$ (indicator of a convex set C) this is projection onto C .

FISTA's claim to fame

Same $O(1/k^2)$ rate as Nesterov, on non-smooth $f + g$ objectives, with no tuning. This is why FISTA became the default for LASSO, matrix completion, compressed sensing, ...

Roadmap

Part 1 — Why Gradient Descent Is Slow

Part 2 — Heavy-Ball Momentum (Polyak, 1964)

Part 3 — Nesterov Acceleration (1983)

Part 4 — Stochastic Gradient Descent

Part 5 — Combining the Ideas

When the full gradient is the bottleneck

A typical ML loss

$$f(x) = \frac{1}{N} \sum_{i=1}^N f_i(x).$$

f_i : per-example loss. N can be 10^6 to 10^9 — ImageNet has $\sim 10^6$; web-scale LMs train on $\sim 10^{12}$ tokens.

The cost of a full gradient

$$\nabla f(x) = \frac{1}{N} \sum_i \nabla f_i(x).$$

Cost per iteration: $O(N)$. At $N = 10^9$ and 10^4 iterations, that's 10^{13} gradient evaluations. For most problems, flatly infeasible.

The stochastic idea

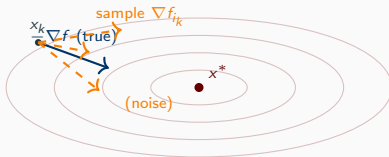
Pick a random $i_k \in \{1, \dots, N\}$. Replace the true gradient with the single-sample estimate $\nabla f_{i_k}(x)$. *In expectation* it equals ∇f . Cost per iteration: $O(1)$. You can afford *many more iterations per unit wall-clock*.

The SGD update — and why it's noisy

$$x_{k+1} = x_k - \alpha_k \nabla f_{i_k}(x_k), \quad i_k \sim \text{Unif}\{1, \dots, N\}.$$

```
for k in range(max_iter):  
    i = np.random.randint(N)           # pick one sample  
    x = x - alpha_k * grad_f_i(x, i)  # one-sample step
```

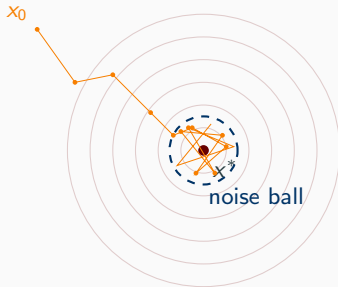
What the stochastic step looks like geometrically



The variance penalty

Each step is $\nabla f(x_k) + (\text{zero-mean noise})$. The noise has variance $\sigma^2 = \mathbb{E}\|\nabla f_i(x) - \nabla f(x)\|^2$,

SGD trajectories — noise ball around the optimum



Two takeaways from the picture

- Early on (x_k far from x^*) the signal dominates, SGD looks like noisy GD.
- Late (x_k near x^*) the noise dominates, iterates bounce around in the noise ball. Reducing α shrinks the ball.

Step-size schedules and mini-batches — the two levers

Diminishing step sizes (Robbins–Monro, 1951)

Pick α_k with $\sum \alpha_k = \infty$ and $\sum \alpha_k^2 < \infty$. Canonical choice: $\alpha_k = c/(k+1)$.

- *Infinite sum*: ensures you keep moving.
- *Finite square-sum*: shrinks the noise ball to $\{x^*\}$.

Converges (in expectation, and a.s.), but slowly: $O(1/k)$ for strongly convex, $O(1/\sqrt{k})$ for smooth convex.

Mini-batches: average b samples per step

$$\hat{g}_k = \frac{1}{b} \sum_{i \in B_k} \nabla f_i(x_k).$$

Variance drops by $1/b$. Cost per step grows by b . Same algorithm, better signal-to-noise.

The sweet spot

$b \in [32, 512]$ for most supervised learning; huge b loses the “cheap iterations” advantage and rejoins deterministic GD’s slow land. Modern deep-learning pipelines use $b \approx 256$ – 1024 with

Roadmap

Part 1 — Why Gradient Descent Is Slow

Part 2 — Heavy-Ball Momentum (Polyak, 1964)

Part 3 — Nesterov Acceleration (1983)

Part 4 — Stochastic Gradient Descent

Part 5 — Combining the Ideas

SGD with momentum — the workhorse for neural nets

Structurally identical to heavy-ball; *semantically* very different, because the gradient is now noisy.

```
v = np.zeros_like(x)
for k in range(max_iter):
    i = np.random.randint(N)
    g = grad_f_i(x, i)           # stochastic gradient
    v = beta * v - alpha * g
    x = x + v
```

Why momentum helps even when the gradient is noisy

The velocity $v_k = -\alpha \sum_{j \leq k} \beta^{k-j} \hat{g}_j$ is an *exponential moving average of past stochastic gradients*. Each sample's noise gets diluted. You get de-noising for free — the same trick you'd use to smooth out a financial time series.

Typical hyperparameters

- $\beta \in [0.9, 0.99]$ — this is momentum=0.9 in `torch.optim.SGD`.

Adam — momentum plus per-coordinate scaling

The update, line by line

```
m = beta1 * m + (1 - beta1) * g           # 1st moment (momentum)
v = beta2 * v + (1 - beta2) * (g * g)      # 2nd moment (Jacobi scale)
m_hat = m / (1 - beta1**t)                 # bias correction
v_hat = v / (1 - beta2**t)
x = x - alpha * m_hat / (np.sqrt(v_hat) + eps)
```

Defaults (Kingma–Ba 2014): $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

What Adam actually is

- m_t is heavy-ball momentum.
- $\sqrt{v_t}$ approximates a *diagonal Hessian scaling*. Coordinates with consistently large gradients get down-weighted.
- So: Adam = momentum + diagonal preconditioning = “momentum plus Jacobi.”

Convergence rates at a glance

Method	Smooth convex	L -smooth, μ -strongly convex
GD	$O(1/k)$	$(1 - 1/\kappa)^k$
Heavy-ball	(no general proof)	$(1 - 1/\sqrt{\kappa})^k$ <i>on quadratics</i>
Nesterov / FISTA	$O(1/k^2)$	$(1 - 1/\sqrt{\kappa})^k$
SGD (diminishing α)	$O(1/\sqrt{k})$	$O(1/k)$ (expected)
SGD + momentum	same order as SGD	same order as SGD
Adam	heuristic	heuristic

The punchline

Accelerated methods square-root the condition number — that's *the* first-order improvement of the last 40 years. Stochastic methods slow the per-step rate but let you do orders of magnitude more steps per unit time. They occupy complementary regimes.

On smooth convex, no first-order method can beat Nesterov's $O(1/k^2)$ in the worst case

The unifying theme — variance reduction

Each of today's three ideas is a variance-reduction trick

- **Nesterov acceleration** reduces variance introduced by *curvature* (evaluating ∇f at a lookahead point corrects in-flight overshoot).
- **Momentum** reduces variance in *update directions* by averaging across iterations — this is true in both the deterministic heavy-ball case and the stochastic SGD case.
- **Mini-batches** reduce variance across *samples* by averaging within an iteration.

Practical recipe

1. **Smooth convex deterministic** $\longrightarrow$ **Nesterov**. Needs L -smoothness (or backtracking line search).
2. **Composite deterministic** (f smooth + g prox-friendly) $\longrightarrow$ **FISTA**.
3. **Stochastic non-convex** (deep learning) $\longrightarrow$ **SGD with momentum**, $\beta = 0.9$, cosine schedule. Switch to **Adam** only if training is unstable.

Summary — what to remember

Three headline results

1. Gradient descent contracts at rate $1 - 1/\kappa$. Accelerated methods (heavy-ball, Nesterov) contract at rate $1 - 1/\sqrt{\kappa}$ — orders of magnitude faster on ill-conditioned problems.
2. Nesterov's $O(1/k^2)$ is the *optimal first-order rate* on smooth convex problems. FISTA extends it to composite (smooth + prox-friendly) problems.
3. Stochastic gradients trade a slower per-step rate for dramatically cheaper steps — the right tool when N is huge. Momentum survives the noise; mini-batching shrinks it.

Further reading

- Nesterov, *Introductory Lectures on Convex Optimization* (2004), Ch. 2.
- Beck & Teboulle, *SIAM J. Imaging Sci.* (2009): the original FISTA paper.
- Bottou, Curtis, Nocedal, *Optimization Methods for Large-Scale Machine Learning*, *SIAM Review* (2018) — the canonical SGD survey.